Sesión 5

Implementación de Arquitecturas Serverless para LLMs

Integración de funciones · Orquestación con colas y triggers · Cold start y caché · Evaluación de desempeño



Diseño de Infraestructura Escalable para LLMs · Maestría · BSG INSTITUTE· Duración: 3 h

Resultados de aprendizaje

1 Integrar funciones

Diseñar la integración de AWS Lambda, Azure Functions y Cloud Run con endpoints de modelos, justificando límites de timeout y concurrencia.

2 Orquestar inferencia

Diseñar flujos con colas y triggers: reintentos, idempotencia, DLQ y backpressure.

3 Mitigar y cachear

Explicar y aplicar mitigaciones de cold start y caché de respuestas, con sus riesgos de seguridad.

4 Evaluar

Definir SLIs/SLOs y un experimento reproducible de desempeño y costo serverless.

Agenda y distribución del tiempo (180 min)

Bloque	Contenido	Min	Tipo
1	Fundamentos serverless y desajuste con LLMs	25	Teoría
2	Integración de funciones: Lambda, Functions, Cloud Run	25	Teoría + demo 1
3	Orquestación: colas, triggers, agentes serverless	35	Teoría + demo 2
4	Cold start y caché de respuestas	30	Teoría + demo 3
5	Seguridad y evaluación de desempeño / FinOps	35	Teoría + demo 4
6	Caso, discusión y cierre	30	Discusión y evaluación

Idea clave 65 % teoría · 25 % demostraciones · 10 % discusión. Las demos usan etiquetas E5.x del repositorio.

Hilo conductor: SoporteIA

E5.2

1 Carga

Picos de inicio de mes, noches casi vacías. Interactiva + por lotes.

2 Restricciones

Datos de clientes (privacidad), presupuesto variable, equipo pequeño de operación.

3 Preguntas de diseño

¿Síncrono o cola? ¿Cuánto pagar por evitar cold start? ¿Qué cachear sin filtrar datos?

Idea clave Cada técnica de hoy se justifica contra este escenario, no por moda tecnológica.

¿Qué es serverless realmente?

1 Sin gestión de servidores

El proveedor aprovisiona, parchea y opera la infraestructura.

2 Escala automática (y a cero)

La capacidad sigue a la demanda; sin demanda, sin instancias.

3 Pago por uso

Se factura por ejecución, tiempo o tokens, no por capacidad ociosa.

4 Dirigido por eventos

HTTP, colas, temporizadores o cambios de datos disparan la ejecución.

Idea clave Escalar a cero ahorra dinero y crea la latencia de arranque: es el origen del cold start.

Responsabilidad compartida en serverless

Capa	IaaS (VM)	Contenedores serverless	Funciones (FaaS)	Inferencia administrada
Hardware y red física	Proveedor	Proveedor	Proveedor	Proveedor
SO y runtime	Cliente	Proveedor (imagen: cliente)	Proveedor	Proveedor
Código de aplicación	Cliente	Cliente	Cliente	Cliente (cliente de la API)
Modelo y pesos	Cliente	Cliente	Cliente / proveedor	Proveedor
Identidad, datos, prompts	Cliente	Cliente	Cliente	Cliente

Idea clave Cuanto más administrado, menos control: IAM, datos y prompts siempre son tuyos.

Tres formas de hacer LLM serverless

Aspecto	FaaS	Contenedor serverless	Inferencia administrada
Ejemplos	Lambda, Azure Functions, Cloud Run functions	Cloud Run, Container Apps	Bedrock, Foundry, Vertex/Gemini
Aloja el modelo	No (llama a uno)	Sí (CPU; GPU según región/cuota)	Lo aloja el proveedor
Control	Código	Código + imagen + concurrencia	Parámetros de la API
Cold start típico	Runtime + dependencias	Imagen + carga del modelo	Gestionado por el proveedor
Mejor para	Orquestación, integración	Modelos pequeños/propios	Máxima calidad con mínima operación

Idea clave Casi siempre se combinan: FaaS orquesta, un servicio administrado infiere.

Por qué los LLMs tensionan el modelo serverless

Supuesto serverless	Realidad LLM	Consecuencia de diseño
Ejecuciones cortas (ms)	Respuestas de segundos a minutos	Streaming o patrón asíncrono con cola
Poca memoria por instancia	Pesos de GB (si se autogestiona)	Inferencia administrada o contenedores con GPU
Escala a cero sin dolor	Cargar pesos tarda segundos	Mitigación de cold start (con costo)
Costo despreciable por petición	Costo por token no trivial	Caché, límites de tokens, FinOps
Sin estado	Contexto y KV cache	Estado externo, sesiones, afinidad

Idea clave Cada desajuste de la tabla es un patrón que estudiaremos hoy.

Anatomía de una petición LLM: TTFT y TPOT



Latencia total ≈ cold start + espera + TTFT + TPOT × tokens_salida + post-proceso

Idea clave Mide por etapas: sin separar TTFT, TPOT y espera no sabrás qué optimizar.

Arquitectura de referencia A1: síncrona

E5.2



Sin slot libre → 429 + Retry-After

- Adecuada para interacción en tiempo real con respuestas cortas.
- Límites de timeout del gateway acotan la duración útil.
- Para trabajos largos, use la arquitectura A2 (con cola).

¿Serverless es la decisión correcta?

Criterio	Favorece serverless	Favorece capacidad dedicada
Patrón de tráfico	Irregular, con valles	Alto y constante
Latencia	Tolera variabilidad ocasional	Requiere latencia muy estable
Operación	Equipo pequeño	Equipo SRE/MLOps maduro
Modelo	Administrado o pequeño	Propio, grande, con GPU
Costo	Variable, por uso	Fijo amortizado por alto uso

Idea clave Decisión basada en tráfico, latencia y operación, justificada con números (ver FinOps).

Integración con AWS Lambda

E5.5.3

1 Ciclo de vida

INIT → INVOKE → SHUTDOWN. Clientes SDK fuera del handler.

2 Mitigaciones

SnapStart (runtimes soportados) y provisioned concurrency.

3 Disparadores

API Gateway / Function URL, SQS, EventBridge, S3.

4 Con Bedrock

Converse API; IAM de mínimo privilegio sobre el modelo.

Idea clave Inicializa fuera del handler; usa cola para trabajos largos; verifica límites vigentes.

Integración con Azure Functions

E5.6.3

1 Flex Consumption

Escala serverless, instancias siempre listas opcionales y VNet.

2 Triggers

HTTP, Service Bus, Event Hubs, colas de Storage.

3 Identidad

Managed Identity + RBAC; sin claves en configuración.

4 Estado

Durable Functions para flujos largos con checkpoints.

Idea clave Identidad administrada + RBAC elimina secretos de larga vida en la función.

Integración con Cloud Run y Cloud Run functions

E5.7.3

1 Contenedor

Cualquier runtime; concurrencia por instancia configurable.

2 Arranque

Instancias mínimas y CPU boost para reducir latencia inicial.

3 Eventos

Pub/Sub y Eventarc disparan funciones/servicios.

4 GPU (según cuota)

Inferencia autogestionada serverless; cold start de modelo.

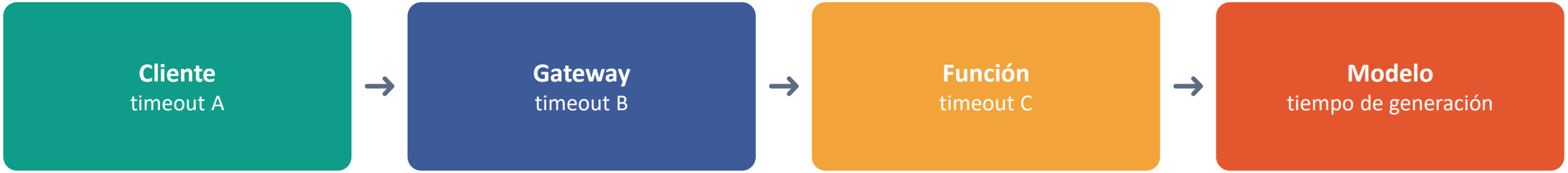
Idea clave La concurrencia por instancia es una palanca de costo, pero cada petición LLM retiene el slot.

Comparación multicloud de la capa de funciones

Aspecto	AWS Lambda	Azure Functions (Flex)	Cloud Run / functions
Unidad de despliegue	Función (zip/imagen)	Function app	Contenedor / función
Concurrencia	1 petición por entorno	Configurable por instancia	Configurable por instancia
Mitigación de arranque	SnapStart, provisioned concurrency	Always ready instances	Min instances, CPU boost
Cola nativa	SQS (event source mapping)	Service Bus trigger	Pub/Sub / Eventarc
Identidad hacia el modelo	Rol IAM	Managed Identity + RBAC	Service Account + IAM

Idea clave Mismo patrón, tres vocabularios: aprende el patrón, no el nombre del servicio.

Timeouts y duración: el problema oculto



- Gana el límite más corto: diseña de afuera hacia adentro.
- Respuesta larga → streaming, o asíncrono (202 + job_id).
- Siempre fija un máximo de tokens de salida.

Idea clave Si la generación puede exceder un timeout de la cadena, cambia de patrón; no solo subas el límite.

Streaming (SSE): latencia percibida vs. control

E5.2

Ventaja	Costo / riesgo
Mejora la latencia percibida (TTFT)	No se puede validar la salida completa antes de enviarla
Mantiene conexión durante generación larga	Patrones sensibles pueden partirse entre chunks
Permite cancelar y ahorrar tokens	Algunos gateways/proxies hacen buffering

Idea clave Streaming es una decisión de UX y de seguridad a la vez.

Colas: nivelar la carga entre cliente y modelo

E5.5.3



- Protege cuotas y presupuesto del modelo ante picos.
- Permite reintentos sin que el cliente espere conectado.
- Costo: latencia extra y necesidad de consultar el estado.

Semántica de entrega: at-least-once

E5.5.3

1 At-least-once

Un mensaje puede entregarse más de una vez: el consumidor debe tolerarlo.

2 Visibility timeout

Debe superar el tiempo de procesamiento o aparecerán duplicados.

3 Dead-letter queue

Tras N recepciones el mensaje se aparta para análisis.

4 Partial batch

Reporta solo los mensajes fallidos del lote.

Idea clave Diseña el consumidor asumiendo duplicados, orden no garantizado y fallos parciales.

Idempotencia: ejecutar dos veces = ejecutar una

E5.2

Nivel	Mecanismo	Protege contra
Borde (API)	Cabecera Idempotency-Key por tenant	Reintentos del cliente y doble clic
Worker	Escritura condicional del estado del job	Entregas duplicadas de la cola
Efectos externos	Claves idempotentes en la herramienta destino	Doble ejecución de acciones de un agente

Idea clave Sin idempotencia, cada reintento puede duplicar costo y efectos secundarios.

Reintentos: backoff, jitter y poison messages

1 Backoff exponencial

Espera creciente entre intentos para no saturar al modelo.

2 Jitter

Aleatoriedad que evita reintentos sincronizados (retry storm).

3 Transitorio vs permanente

Reintenta 429/5xx; no reintentes errores de validación.

4 Poison message

Siempre falla: se aísla en DLQ con alarma.

Idea clave Reintentar es un mecanismo de carga: acótalo con límites y presupuesto.

Backpressure y control de concurrencia

E5.4

Escenario (40 clientes, capacidad 4)	RPS	p95 ms	Respuestas
Sin límite de espera	16,5	2412	200: 200
Backpressure (espera ≤ 0,3 s)	101,9	510	200: 32 · 429: 168

- Datos del experimento simulado (proveedor mock): ilustran el mecanismo, no el rendimiento real.
- Falla rápido (429 + Retry-After) para proteger la calidad de lo admitido.

Idea clave Rechazar a tiempo es una función de calidad, no un fallo.

Triggers y entrega de eventos por proveedor

Aspecto	AWS	Azure	Google Cloud
Servicio de mensajes	SQS	Service Bus	Pub/Sub
Conector a cómputo	Event source mapping	Service Bus trigger	Eventarc / trigger de función
DLQ	Redrive policy	DLQ nativa + MaxDeliveryCount	Dead-letter topic
Control de concurrencia	MaximumConcurrency	maxConcurrentCalls + máx. instancias	Máx. instancias

Idea clave Mismo patrón (cola → consumidor → DLQ) con nombres y perillas distintas.

Agentes serverless en las tres nubes

Nube	Consola (sin código)	Código	Runtime serverless
AWS	Bedrock Agents (E5.5.1)	Strands Agents SDK (E5.5.2)	AgentCore Runtime
Azure	Portal de Foundry (E5.6.1)	Microsoft Agent Framework, otros (E5.6.2)	Foundry Agent Service
Google Cloud	Agent Studio/Designer (E5.7.1)	Agent Development Kit (E5.7.2)	Runtime de agentes / Cloud Run

Idea clave Las tres nubes convergen: modelo + herramientas + runtime administrado + trazas.

Azure: cuatro frameworks agénticos, estados distintos

E5.6.2

Framework	Estado (al preparar el material)	Cuándo elegirlo
Microsoft Agent Framework	Sucesor recomendado (GA)	Proyectos nuevos en Azure/Foundry
Semantic Kernel	En mantenimiento	Código existente; migrar gradualmente
AutoGen	Legado / archivado	Solo estudio y migración
LangGraph	Independiente (LangChain)	Grafos con estado, portabilidad entre nubes

Idea clave Verifica el estado de soporte de un framework antes de adoptarlo: es parte del riesgo de diseño.

Anatomía del cold start en una petición LLM

E5.1



- Cada capa tiene su propia mitigación y su propio costo.
- En inferencia administrada, las dos últimas las absorbe el proveedor.

Idea clave No existe 'el' cold start: es una suma de capas que se mide y se mitiga por separado.

Estrategias de mitigación de cold start

1 Init ansiosa (eager)

Carga al arrancar, no en la primera petición.

2 Snapshots

SnapStart: restaura un entorno ya inicializado.

3 Instancias calientes

Min instances / always ready / provisioned concurrency.

4 CPU boost

Más CPU durante el arranque.

5 keep_alive

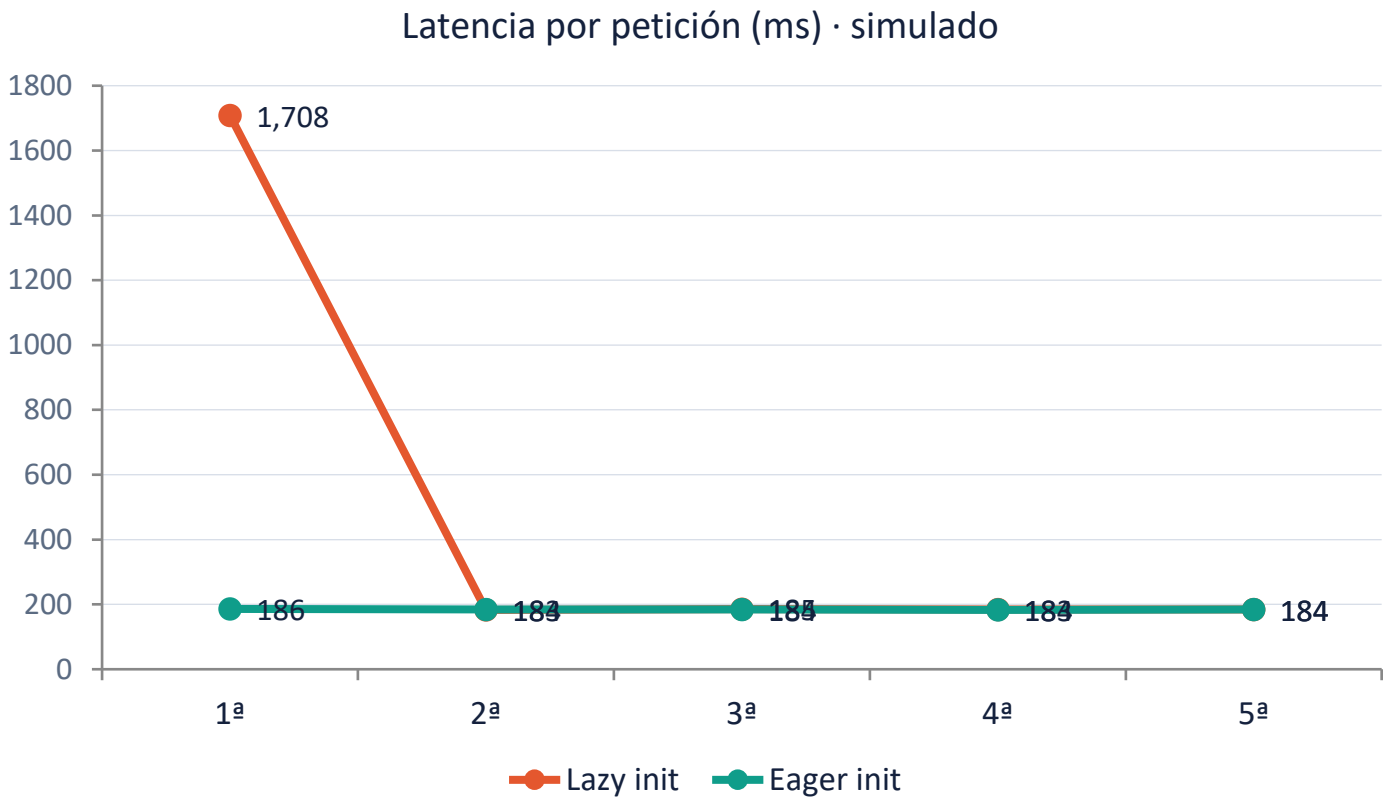
El servidor de inferencia mantiene el modelo cargado.

6 Pesos en la imagen

Evita descargar; imagen grande.

Evidencia E1: inicialización perezosa vs. ansiosa

E5.4



- Lazy: la primera petición paga 1,5 s de carga.
- Eager: el costo se traslada al arranque del entorno.
- Datos simulados (mock): repetir con Ollama real (E5.1).

Idea clave Mitigar no elimina el costo: lo mueve a un momento menos dañino o lo paga como reserva.

El costo de mitigar: reserva fija vs. latencia

1 Costo de la reserva

k × tarifa_instancia × horas. Fijo, aunque no haya tráfico.

2 Beneficio

Menos peticiones con cold start: mejor p95/p99 y experiencia.

3 Regla

Reserva solo donde el SLO o el valor del usuario lo justifiquen; el resto, init ansiosa y diseño asíncrono.

Idea clave Rellena las fórmulas con precios vigentes del proveedor; este material no los inventa.

Caché de respuestas: qué, cuándo y cómo

E5.2

1 Determinismo

Solo temperature=0 y mismos parámetros del modelo.

2 Clave

tenant + modelo + parámetros + prompt normalizado (hash).

3 TTL y LRU

Vigencia de datos y límite de memoria.

4 Single-flight

Peticiones idénticas simultáneas comparten un solo cómputo.

Idea clave Cachea solo lo determinista y siempre dentro del límite del tenant.

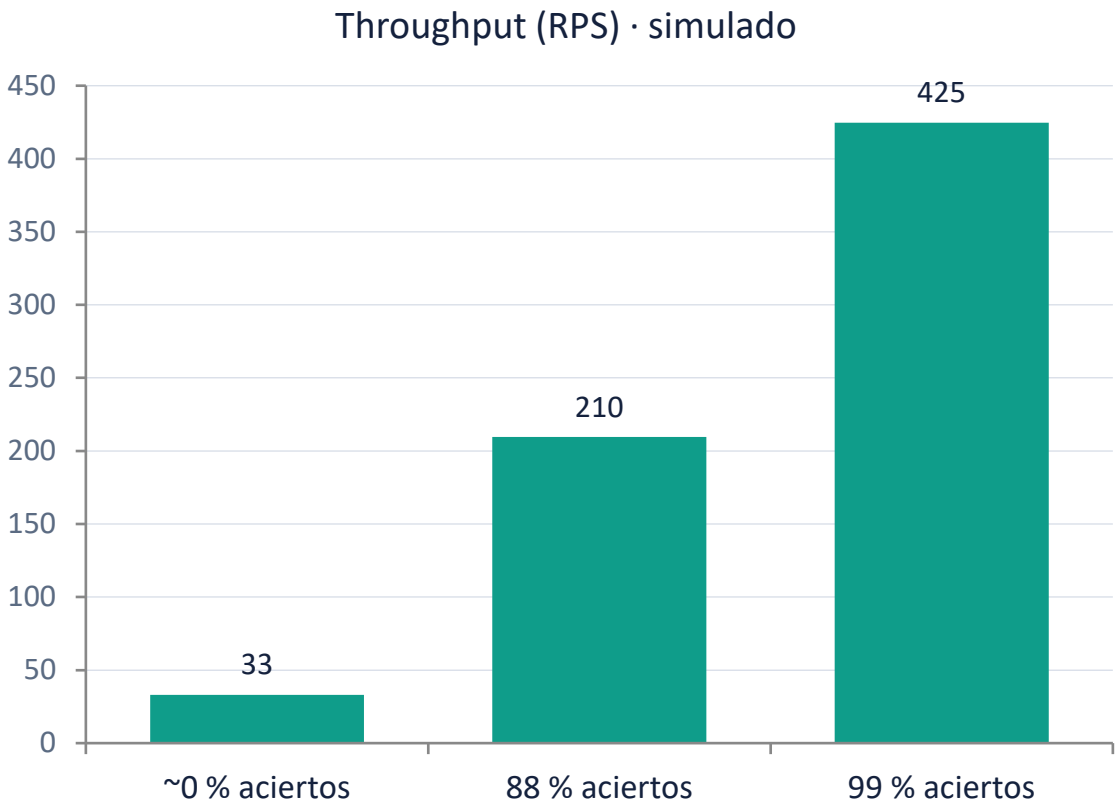
Riesgos de seguridad del caché

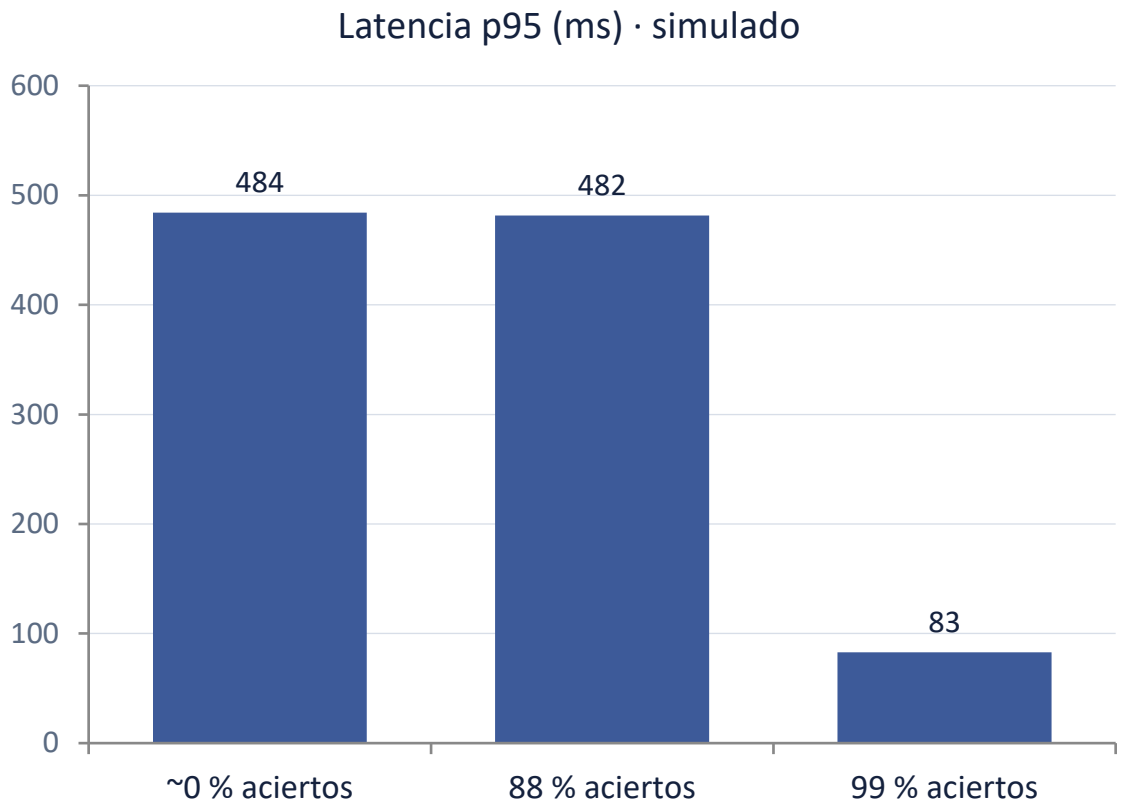
Riesgo	Causa	Control
Fuga entre tenants	Clave sin identificador de tenant	Tenant en la clave + prueba automatizada
Obsolescencia	TTL largo o sin invalidación	TTL por tipo de dato + invalidación
Datos personales persistidos	Respuestas con PII en caché	No cachear respuestas con PII; cifrar; retención
Caché envenenado	Salida manipulada por inyección	Validar antes de cachear; TTL corto

Idea clave El caché es un almacén de datos nuevo: aplícale las mismas reglas de seguridad.

Evidencia E2: efecto del caché sobre throughput y p95

E5.4





Idea clave El valor del caché depende de la tasa real de repetición: mídela antes de prometer ahorros.

Superficie de ataque de una arquitectura serverless LLM

E5.8

Componente	Amenaza principal	Control
Entrada (prompt)	Inyección de prompt directa e indirecta	Validación, separación de privilegios, guardrails
Salida	Manejo inseguro (HTML, imágenes externas)	Sanitización, escape, CSP
Cola y caché	Mensajes manipulados, fuga entre tenants	Autorización, clave por tenant, cifrado
Herramientas / agentes	Agencia excesiva	Mínimo privilegio, aprobación humana, límites
Costo	Denial of wallet	Límites de tokens, rate limit, presupuestos

Idea clave Denial of wallet: en serverless el abuso se traduce en facturas, no solo en caídas.

Defensa en profundidad en el gateway

E5.8



- Logs con huellas (hash), nunca prompts completos.
- Las heurísticas son una capa: no sustituyen guardrails gestionados.

Idea clave Cada capa asume que la anterior puede fallar.

Identidad, secretos y red: lo que más falla en la práctica

1 Mínimo privilegio

Un rol por función, acotado al modelo y a la cola concretos.

2 Sin claves de larga vida

Managed Identity, roles IAM o Service Accounts; secretos en gestor con rotación.

3 Red

Endpoints privados y egreso restringido cuando esté disponible.

4 Cifrado

En tránsito y en reposo: colas, tablas y cachés.

Idea clave Los comodines (Resource: *) son deuda de seguridad: los labs los marcan para que los elimines.

Agentes y herramientas: riesgos añadidos

1 Agencia excesiva

Pocas herramientas, cada una con permisos propios y mínimos.

2 Inyección indirecta

El contenido de herramientas y documentos son datos, no instrucciones.

3 Acciones sensibles

Aprobación humana e idempotencia en efectos externos.

4 Bucles y costo

Límite de pasos, de tokens y de tiempo por ejecución.

Idea clave Cuando el modelo puede actuar, la seguridad se diseña como en cualquier sistema con privilegios.

Métricas de evaluación serverless para LLMs

E5.4

Familia	Métricas
Latencia	TTFT, TPOT, latencia total (p50/p95/p99)
Serverless	Tasa de cold start, duración de init, concurrencia usada
Colas	Edad del mensaje más antiguo, profundidad, profundidad de DLQ
Eficiencia	Tasa de aciertos de caché, tokens por petición
Salud y costo	Errores, 429, costo por petición

Idea clave Usa percentiles, nunca solo promedios: la experiencia real vive en la cola de la distribución.

De métricas a SLI, SLO y presupuesto de error

1 SLI

Qué se mide: p. ej., % de peticiones con TTFT bajo el umbral.

2 SLO

Objetivo en una ventana: p. ej., ≥ 95 % durante 30 días.

3 Presupuesto de error

1 − SLO: guía cuánto riesgo se puede asumir al desplegar.

Idea clave Un SLO por tipo de carga: interactivo y por lotes tienen necesidades distintas. Umbrales: hipótesis a validar.

Cómo evaluar bien: diseño del experimento

E5.4

1 Separar frío y caliente

Mide la primera petición tras inactividad aparte del régimen estable.

2 Controlar variables

Mismo prompt, tokens de salida, región y configuración.

3 Percentiles y repeticiones

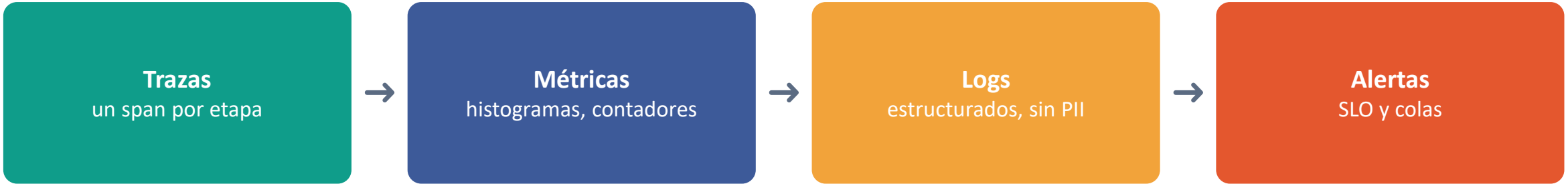
p50/p95/p99 con varias corridas, no un solo promedio.

4 Carga realista

Cuidado con la omisión coordinada en pruebas de ciclo cerrado.

Idea clave Si no es reproducible, no es evidencia: registra versión, región, configuración y fecha.

Observabilidad: trazas, métricas y logs



- Propaga el trace id dentro del mensaje de la cola.
- Estándar abierto (OpenTelemetry) + servicio nativo de cada nube.

Idea clave Sin trazas correlacionadas, depurar un flujo asíncrono es adivinar.

FinOps: estimar y comparar costos

Concepto	Fórmula
Costo por petición LLM	tokens_ent × P_ent + tokens_sal × P_sal
Cómputo serverless	GB·s × P_gbs + invocaciones × P_inv
Con caché (acierto h)	(1 − h) × costo_LLM + h × costo_caché
Punto de equilibrio	N* = costo_fijo_mensual / costo_variable_por_petición

Idea clave Las fórmulas son del curso; los precios los trae cada estudiante de la página oficial del día.

Caso integrador: decisiones para SoporteIA

E5.2

ADR	Decisión	Trade-off aceptado
001	Cola + DLQ + idempotencia para trabajo no interactivo	Latencia extra y consulta de estado
002	Caché solo con temperature=0, clave por tenant, TTL	Riesgo de obsolescencia; sin caché semántico
003	Cold start por capas; reserva solo si el SLO lo exige	Costo fijo puntual vs latencia de cola

Idea clave Una arquitectura es un conjunto de decisiones con sus trade-offs explícitos.

Mapa de demostraciones y laboratorios

ID	Demo / laboratorio	Verificación al preparar
E5.1 · E5.2 · E5.3	Ollama local · gateway FastAPI · React	Mock probado; Ollama real pendiente
E5.4 · E5.8	Carga y experimentos · pruebas de seguridad	Ejecutado (simulado) · 51 tests
E5.5.x (AWS)	Bedrock Agent · Strands/AgentCore · SQS→Lambda	cfn-lint · compilación · sin nube
E5.6.x (Azure)	Portal Foundry · 4 frameworks · Service Bus→Functions	Construcción offline · sin nube
E5.7.x (GCP)	Consola · ADK · Pub/Sub→Cloud Run · GPU	Importación · sin nube

Idea clave Todo lo marcado 'pendiente' debe ejecutarse en una cuenta de prueba antes de la clase.

Errores frecuentes de diseño

1 Caché sin tenant

Fuga de datos entre clientes.

2 Visibility timeout corto

Duplicados y doble costo.

3 Reintentar todo

Amplifica fallos y errores permanentes.

4 Sin límite de tokens

Denial of wallet.

5 Un solo filtro

Se evade con una variante.

6 Medir promedios

Oculta la cola p95/p99.

Discusión y preguntas de análisis

1 ¿Cola o síncrono?

¿Qué cargas de SoporteIA justifican una cola y qué pagas por ello?

2 Cold start

¿Cuándo vale la pena reservar instancias? ¿Cómo lo demostrarías con datos?

3 Caché y privacidad

¿Qué respuestas jamás cachearías? ¿Cómo pruebas el aislamiento?

4 Break-even

¿A qué utilización conviene capacidad dedicada frente a serverless?

Conclusiones

1 Diseñar para LLM

Latencia, estado y costo por token cambian las reglas serverless.

2 Cola + idempotencia

Convierten trabajo largo en procesos fiables.

3 Mitigar con evidencia

Cold start es una decisión económica.

4 Caché con límites

Tenant, TTL y no persistir PII.

5 Evaluar bien

Percentiles, frío/caliente, reproducibilidad.

Referencias

- AWS Lambda, SQS, Bedrock y AgentCore: docs.aws.amazon.com · Strands Agents: strandsagents.com
- Azure Functions, Service Bus y Microsoft Foundry: learn.microsoft.com/azure · Microsoft Agent Framework: learn.microsoft.com/agent-framework
- Google Cloud Run, Pub/Sub y plataforma de agentes: cloud.google.com/docs · Agent Development Kit: google.github.io/adk-docs
- OWASP GenAI Security Project (Top 10 LLM y Agentic): genai.owasp.org
- NIST AI RMF 1.0 (AI 100-1) y AI 600-1: nist.gov/itl/ai-risk-management-framework · ISO/IEC 42001:2023
- Patrones de nube (Queue-Based Load Leveling, Competing Consumers): learn.microsoft.com/azure/architecture/patterns
- Repositorio del curso: docs/referencias.md (lista completa y fecha de verificación)